

Q-RouteAI

Quantum-Inspired Intelligent Traffic Route Optimization Platform

Detailed Solution: Innovation · Architecture · Feasibility · Viability · Challenges · Impact

PS ID

SIH26137

THEME

Transportation & Logistics

ORGANIZATION

Egreen Quanta

Problem: Quantum-Inspired Intelligent Traffic Route Optimization in Transportation Systems Using Metaheuristic Optimization

CONTENTS

1 Executive Summary	2
2 Problem Understanding & Scope	3
3 Proposed Solution	4
4 Core Algorithm: AS-QPSO-SA-M	5
5 Innovation & Uniqueness	7
6 System Architecture	8
7 Flow Chart	10
8 Technology Stack	11
9 Prototype Validation & Benchmark Plan	12
10 Feasibility	14
11 Viability	15
12 Potential Challenges & How We Tackle Them	16
13 Impact & Benefits	17
14 Roadmap & Future Scope	17
References	18
Appendix: What Changed from Blueprint v1	18

1 Executive Summary

Q-RouteAI is a full-stack routing platform built for the Egreen Quanta problem statement (SIH26137). It models a city as a time-dependent weighted road graph, reduces a fleet's job to a cost matrix between stops, and solves the genuinely hard part — *sequencing stops and assigning them to vehicles* (an NP-hard vehicle routing problem) — with a quantum-inspired metaheuristic engine, **AS-QPSO-SA-M** (Adaptive-Superposition Quantum-behaved PSO + Simulated Annealing + Memetic search). The engine is **quantum-inspired, not quantum hardware**: it borrows the wave-function / delta-potential-well sampling law of QPSO and runs on ordinary CPUs (GPU optional).

Design principle: Construct → Improve → Repair.

1 Construct a strong starting plan in well under a second (Clarke-Wright savings on a road-network cost matrix). **2 Improve** it inside a fixed time budget with the quantum-inspired engine. **3 Repair** it live when a road closes or an order arrives (*Local Wave Recollapse*: warm-start, re-optimize only the affected routes).

What an evaluator sees in the live demo

- **Race Arena** — the same instance and the same time budget, side by side: greedy, GA, PSO and the quantum-inspired engine, with live convergence curves and a distance / time / CO₂ scoreboard.
- **Click-to-disrupt** — click any road, mark it blocked or slow; affected routes are repaired live and pushed to the map over WebSocket.
- **Speed ↔ Eco slider** — the fitness function shifts between time, distance and CO₂; every route carries an explanation of why it looks the way it does.
- **One-click benchmark report** — Solomon / Gehring-Homberger / CVRPLIB instances, fixed seeds, statistical tests, exportable tables.

Evidence, not slogans. We prototyped the engine and raced it against greedy, GA and PSO on synthetic routing instances (50-400 stops, equal time budgets, one CPU core). Three findings shaped the design: (i) random-start swarms lose to a one-second savings heuristic from 100 stops upward; (ii) starting from that heuristic and improving it saves a further 0.9-2.8 % of distance; (iii) our quantum-inspired jump operator edged out a plain GA by 0.2-0.3 percentage points, while local search contributes most of the gain. We report these results as measured, including where we do not win (Section 9).

Key numbers at a glance

Item	Value
Problem	Multi-vehicle routing with capacity and time windows (NP-hard), dynamic re-routing, CO ₂ -aware objective
Core engine	AS-QPSO-SA-M: seeded population → quantum-inspired exponential-jump moves → SA acceptance → windowed local search
Runs on	CPU only (NumPy + Numba); optional GPU (PyTorch) for very large swarms
Stack	Next.js + MapLibre · FastAPI + WebSocket · PostGIS + Redis · Docker Compose
Prototype evidence	10-17 % shorter than nearest-neighbour dispatch and 0.9-2.8 % shorter than a Clarke-Wright plan (50-400 stops, synthetic instances, single CPU core, 3-14 s budgets)
Build effort	Engine core already prototyped; 36-hour plan with a clear MVP cut-line (Section 10)

2 Problem Understanding & Scope

2.1 What the problem statement really asks

City routing and delivery systems struggle because the number of possible stop orders and vehicle assignments explodes combinatorially. True quantum computers are not ready for widespread use, so the statement asks for a *quantum-inspired* software approach that solves large routing problems faster and better than conventional heuristics, and that can be benchmarked against them. We decompose this into three coupled problems:

#	Sub-problem	Why it is hard	Q-RouteAI answer
P1	Static fleet routing (CVRP / VRPTW): which stops go to which vehicle, in which order, within capacity and time windows	NP-hard; exact methods stop scaling at a few hundred stops; unseeded metaheuristics converge slowly	Construct → Improve pipeline with AS-QPSO-SA-M (Sections 4, 6)
P2	Dynamic re-routing : roads close, congestion shifts, new orders arrive	Full re-solve from scratch is too slow and produces disruptive plan changes	Local Wave Recollapse: warm start, affected routes only (Section 4.7)
P3	Sustainability-aware objective : fuel and CO ₂ depend on speed, load and stop-and-go traffic	Shortest is not greenest; a single weight cannot serve all fleets	Speed ↔ Eco slider over a speed- and load-dependent emission model (Sections 4.1-4.2)

2.2 A key architectural insight

Shortest path is not the hard problem; sequencing is. A point-to-point shortest path is solvable in polynomial time and Dijkstra / A* are *optimal* for it, so no metaheuristic can beat them there. Q-RouteAI therefore uses Dijkstra where it is optimal (building the stop-to-stop cost matrix and expanding final routes into road geometry) and uses the quantum-inspired engine only where the problem is NP-hard: deciding the order and assignment of many stops. The optimiser never runs on the raw 10,000-node road graph; it runs on the small graph of depot + stops.

2.3 Stakeholders and what they get

Stakeholder	Need	What Q-RouteAI provides
Egreen Quanta (technology owner)	Credible quantum-inspired routing capability with proof of benefit	Engine + reproducible benchmark harness + Race Arena
Municipal traffic authorities	Understand fleet impact of closures and events; reduce congestion and emissions	What-if closure simulator, edge-load heatmap, city-level CO ₂ view
Logistics & supply-chain enterprises	Lower cost per delivery, on-time performance, ESG reporting	Optimised plans, time-window compliance, per-route CO ₂ report
Delivery fleet managers / dispatchers	Fast, trustworthy daily plans that survive disruptions	One-click optimise, live repair, explainable routes, manual override
Urban commuters (indirect)	Fewer delays, less congestion; multi-stop errands	Less fleet-induced congestion; optional multi-stop trip planner (TSP with time windows)

2.4 Scope and assumptions

In scope (MVP)	Explicitly out of scope / later phase
Homogeneous fleet with capacity; single depot; time windows; time-band traffic weights; synthetic congestion profiles calibrated to city-level indices; OSM road network; web dashboard	Turn-by-turn driver navigation app; heterogeneous fleets and multi-depot (Phase 2); real-time GPS ingestion (adapter provided, pilot data needed); execution on physical quantum hardware (QUBO export only)

3 Proposed Solution

3.1 Solution overview

Q-RouteAI is delivered as eight cooperating modules. The first three build a correct, fast *problem*; modules 4-6 solve and adapt it; modules 7-8 make the result visible and provable.

Module	What it does	Key technology	Jury metric served
M1 Road Graph Builder	Downloads OSM road network for the chosen area, builds a directed graph with length, speed limit and road class; stores it in PostGIS	OSMnx, PostGIS	Real-world applicability
M2 Traffic & Emission Weighting	Attaches a time-of-day congestion multiplier and a speed/load-dependent emission factor to every edge; pluggable live-feed adapter	Python, config-driven profiles	Dynamic traffic handling
M3 Cost-Matrix Service	Multi-source Dijkstra between depot and stops per time band; caches in Redis; invalidates only affected rows/columns on disruption	SciPy csgraph, Redis	Execution time
M4 Optimization Engine	Clarke-Wright construction, then AS-QPSO-SA-M improvement under a time budget; GA, PSO and greedy baselines behind the same interface	NumPy + Numba (PyTorch optional)	Convergence, distance/cost reduction
M5 Dynamic Re-routing	Local Wave Recollapse: warm-started repair of affected routes after a disruption or new order	Same engine, partial problem	Real-time adaptation
M6 Eco & Cost Model	Weights time / distance / CO ₂ ; Pareto archive of plans; per-route fuel and CO ₂ report	Custom model (COPERT / PRP-style)	Cost and carbon reduction
M7 Visual Layer	Map with routes and closures, live convergence charts, Race Arena, disruption clicker, explain panel, scalability dashboard	Next.js, MapLibre GL, D3	UI clarity
M8 Benchmark Harness	Loads Solomon / Gehring-Homberger / X-set, fixed seeds, equal budgets, statistics, CSV/report export	pandas, SciPy stats	Scalability index, credibility

3.2 User journeys

- **Fleet manager.** Uploads a CSV of orders (location, demand, time window) and the fleet definition, sets the Speed ↔ Eco slider, presses *Optimise*. Within the time budget the map shows colour-coded routes, ETAs, load factors and CO₂; the manager can lock a route and re-optimize the rest before dispatch.
- **City traffic authority.** Opens the what-if view, closes a flyover for an event, and immediately sees which fleet routes are affected, where fleet load concentrates (edge-load heatmap) and the change in fleet distance and emissions.
- **Evaluator / jury.** Opens *Race Arena*, picks an instance size, and watches greedy, GA, PSO and AS-QPSO-SA-M work on the same problem with the same budget; then injects a disruption and watches the repair.

3.3 Worked scenario

A courier company runs 8 vans against 120 same-day orders across a metro area. (1) Orders and fleet are imported; M3 builds a 121 × 121 cost matrix from the road graph for the 10:00 time band. (2) M4 constructs a savings-based plan and starts improving it. (3) The dashboard streams the best-so-far cost every few hundred milliseconds; the dispatcher accepts the plan after the time budget. (4) At 11:20 a flyover is closed: the user clicks it, M2 marks the edges blocked, M3 recomputes only the rows and columns whose shortest paths used them, and M5 re-optimises the 2-3 affected routes while the other vans keep their plans. (5) The explain panel reports the change in distance, lateness and CO₂. All figures in this scenario are computed live by the platform; the numbers quoted elsewhere in this document come from the prototype benchmark in Section 9.

4 Core Algorithm: AS-QPSO-SA-M

4.1 Problem model

Let the stop set be $S = \{0, 1, \dots, n\}$ (0 is the depot), with demand q_i , service time s_i and time window $[e_i, l_i]$. The cost matrix gives, for every ordered pair and time band, the shortest-path travel time t_{ij} , distance d_{ij} and CO₂ c_{ij} computed on the road graph. A vehicle has capacity Q . A solution is a set of routes starting and ending at the depot that visits every stop once.

$$F(R) = w_t \cdot T(R)/T_0 + w_d \cdot D(R)/D_0 + w_e \cdot C(R)/C_0 + \lambda_q \cdot \sum \max(0, \text{load} - Q) + \lambda_w \cdot \sum \max(0, \text{arrival}_i - l_i)$$

T, D, C are total travel time, distance and CO₂; the reference values T_0, D_0, C_0 come from the constructed savings plan, so all terms are dimensionless. The Speed ↔ Eco slider $s \in [0, 1]$ sets $w_t = 1-s$, $w_e = s$ (with w_d optional); the Pareto archive keeps the non-dominated plans seen while sweeping s . Hard constraints are handled by the decoder where possible and by penalties otherwise.

4.2 Emission model

For an edge of length d driven at congestion-adjusted average speed v with load fraction ρ : $\varepsilon = d \cdot (a/v + b + c \cdot v^2) \cdot (1 + \kappa \cdot \rho)$. The U-shaped speed term captures high consumption in crawling traffic and at high speed, and the load term captures payload. Coefficients a, b, c, κ are configuration per vehicle class (COPERT / pollution-routing-problem style; Bektaş & Laporte, 2011) and are calibrated later against telematics. We report **relative** savings, which are robust to coefficient error, rather than absolute tonnes.

4.3 From continuous swarm positions to discrete routes

Roads are discrete, so a continuous particle position must be decoded. In **continuous mode** each particle is a vector $x \in \mathbb{R}^n$ (random-key encoding, Bean 1994): sorting its entries gives a customer permutation (a *giant tour*). The giant tour is cut into capacity- and time-window-feasible routes by **Prins' optimal split** (Prins 2004), a shortest-path dynamic programme over the tour that is $O(n \cdot b)$, where b is the maximum number of stops per route. With time windows the split extends one stop at a time while tracking arrival time and stops extending a route as soon as a hard deadline is violated, which is exact because later stops in a fixed order can only arrive later. In **discrete mode** (Section 4.5) particles are permutations directly and only the split is needed.

4.4 QPSO: the wave-function law

In quantum-behaved PSO (Sun, Feng & Xu, 2004) particles have no velocity. Solving the Schrödinger equation for a delta-potential well centred at an attractor p yields a position distribution that is sampled directly. With M particles in n dimensions:

$$\begin{aligned} mbest(t) &= (1/M) \sum_i pbest_i(t) \\ p_{ij} &= \phi_{ij} \cdot pbest_{ij} + (1 - \phi_{ij}) \cdot gbest_j, \phi \sim U(0, 1) \\ x_{ij}(t+1) &= p_{ij} \pm \alpha \cdot |mbest_j - x_{ij}(t)| \cdot \ln(1/u), u \sim U(0, 1), \text{sign chosen with probability } 1/2 \end{aligned}$$

The step $\alpha \cdot L \ln(1/u)$ with $L = |mbest - x|$ is exponentially distributed, so the landing point is Laplace-distributed around p with scale αL : usually close (exploitation) but with a non-zero, analytically known probability $P(|x-p| > cL) = e^{-c/\alpha}$ of a long jump (exploration). α (contraction-expansion coefficient) decays linearly from 1.0 to 0.5. For the basic algorithm, convergence in probability requires $\alpha < e^{\gamma} \approx 1.781$ (Sun et al., 2012), so our adaptive widening of α on stagnation is capped below that bound. The word “tunnelling” is an analogy for these long jumps, not a physical claim.

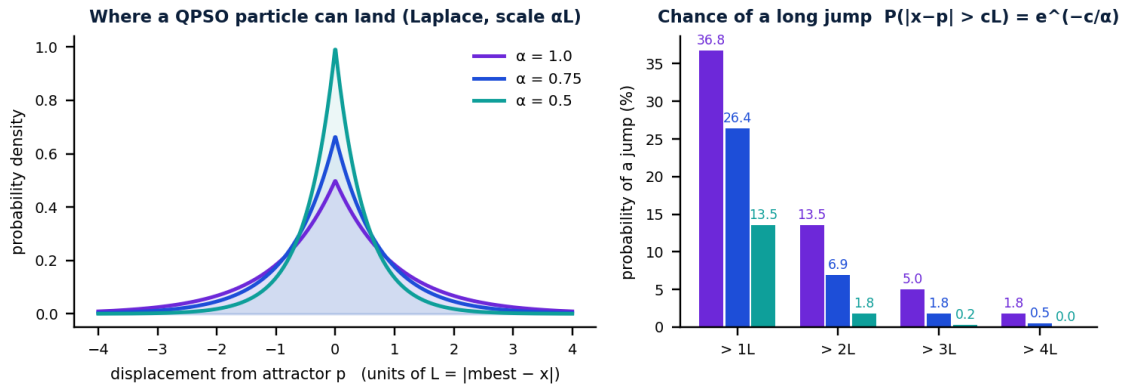


Figure 1: The exponential jump law behind QPSO. Larger α early in the search means frequent long jumps; smaller α late in the search concentrates the swarm.

4.5 Discrete mode: what runs at scale

Our prototype showed that random-key decoding loses locality as n grows (Section 9). The production engine therefore keeps the QPSO ingredients but applies them to permutations:

- **mbest** becomes a *consensus tour*: customers ordered by their mean position over the swarm's best tours.
- **Spread L** is the breakpoint distance between a candidate and the consensus tour (number of adjacent pairs not shared) — large when the swarm is diverse, small when it has converged.
- **Attractor p** is an order crossover (OX) of two parents, a discrete analogue of the random convex combination of pbest and gbest.
- **Wave-function collapse**: apply $k = \text{round}(\alpha \cdot L \cdot \ln(1/u))$ random swap / inversion / relocate moves. The jump size adapts to swarm spread exactly as in the continuous law.
- **Simulated-annealing acceptance**: a child worse than its parent is accepted with probability $\exp(-\Delta F/T)$, with $T = T_0(1 - t/t_{\max})^2$; two elites are always kept.
- **Memetic step**: every 5 generations a windowed 2-opt + Or-opt search improves the best tour (evaluated through the optimal split), limited to a 15 % share of run time so it can never starve the global search.

4.6 Pseudocode

```
ENGINE(instance, time_budget)
  D ← cost_matrix(stops, time_band)           # Layer 1: Dijkstra on the road graph
  g0 ← clarkewright(D)                         # CONSTRUCT (sub-second)
  P ← {g0} ∪ {perturb(g0, k_i)}, |P| = 30      # seeded population
  while time < time_budget:
    # IMPROVE
    cons ← consensus(P)                       # discrete mbest
    new ← two best individuals of P           # elitism
    repeat until |new| = 30:
      a, b ← tournament(P), tournament(P)
      c ← OX(a, b)                           # attractor
      L ← breakpoints(c, cons)                # swarm spread
      k ← round(α · L · ln(1/u)), u ~ U(0,1)
      c ← perturb(c, k)                       # wave-function collapse
      if F(c) > F(a) and rand() > exp(-(F(c)-F(a))/T): c ← a # SA
      new ← new ∪ {c}
    P ← new
    every 5 generations, if LS_time < 15 % of elapsed: best(P) ← 2opt_0ropt(best(P))
    α ← 1.0 - 0.5·t/t_max ; T ← T0·(1 - t/t_max)^2
  return split(best(P)) → routes
```

4.7 Local Wave Recollapse (dynamic repair)

- **Detect** the routes whose paths use a blocked or slowed edge (or that must absorb a new order).
- **Update** only the affected rows and columns of the cost matrix (Redis invalidation) — unaffected routes are frozen.
- **Warm start**: initialise the population as small perturbations around the current best plan restricted to the affected stops, with a small α (0.3-0.5): the wave function “re-collapses” around a known good state instead of starting again.

- **Anytime output:** push a fast first repair, then keep improving until the repair budget ends; merge with frozen routes and stream the result over WebSocket. The latency target (sub-second for up to about 100 affected stops) is a *measured KPI*, not a guarantee.

4.8 Complexity

Per generation the engine costs $O(M \cdot n \cdot b)$ for splitting plus $O(M \cdot n)$ for consensus and breakpoint distances (b = stops per route), i.e. the same order as PSO or GA: QPSO offers **no asymptotic speed-up**, and we do not claim one. Its value is search quality per unit time. The cost matrix costs one Dijkstra per stop (SciPy, cached per time band). The local search is the most expensive part ($O(n \cdot w \cdot n \cdot b)$ per pass for window w); delta evaluation would reduce it to about $O(n \cdot w \cdot b)$ and is the first optimisation planned.

5 Innovation & Uniqueness

Most submissions to this problem statement will combine a map, a Dijkstra call and a textbook PSO. Q-RouteAI differs in *engineering discipline, evidence and system-level features*, not just in the choice of algorithm.

#	Innovation	What is new and why it matters
11	Construct → Improve → Repair pipeline	Derived from our own experiments: unseeded swarms lose to a one-second greedy heuristic at ≥ 200 stops. Seeding with savings and then improving is what makes the metaheuristic useful in practice.
12	Quantum-inspired exponential-jump operator on permutations	Translates the QPSO delta-well law to discrete tours: the jump size is $\alpha \cdot L \cdot \ln(1/u)$ with L measured from the swarm's own consensus tour, so exploration shrinks automatically as the swarm converges.
13	Local Wave Recollapse	Incremental, warm-started repair of only the affected routes; avoids re-solving the whole city and avoids wholesale plan churn for drivers.
14	Eco-Pareto engine	Speed ↔ Eco slider over a speed- and load-dependent emission model, with a Pareto archive and per-route CO ₂ accounting; sustainability is an objective, not a label.
15	Congestion-balancing layer (stretch)	Penalises edges that the fleet itself would overload, so many vehicles do not all take the same “best” road and create the congestion they try to avoid.
16	Race Arena + reproducible benchmark harness	Same instance, same budget, fixed seeds, statistical tests, and results reported even where we lose. Trust through evidence.
17	Explainable routes	“Why this route?” panel: cost breakdown, binding capacity / time-window constraints, and what the alternative would cost. Builds dispatcher trust.
18	Quantum-ready adapter (stretch)	Exports small clusters (≤ 12 stops) as QUBO problems solved by a simulated-annealing sampler today and annealer-compatible tomorrow; a credible path from quantum-inspired to quantum-enabled.
19	Offline-first, CPU-only, open stack	Runs from a single docker-compose on a laptop with cached OSM data; no paid API or token needed — affordable for Indian SMEs and safe for live demos.

How Q-RouteAI compares

Capability	Map-app style routing (Dijkstra / A*)	Classical VRP solvers (e.g. OR-Tools)	Generic PSO / GA demo apps	Q-RouteAI
Multi-vehicle, capacity, time windows	No	Yes	Partial	Yes
Quantum-inspired search	No	No	No	Yes
Warm-start live repair after disruption	Re-query only	Re-solve (warm start possible)	Rare	Yes (Local Wave Recollapse)
Eco / CO₂ objective with slider	Limited	Custom coding needed	Rare	Built in
Transparent live benchmark vs baselines	No	No	Rare	Built in (Race Arena)

OR-Tools-class solvers are strong; we use them as an external benchmark reference, not as a straw man.

6 System Architecture

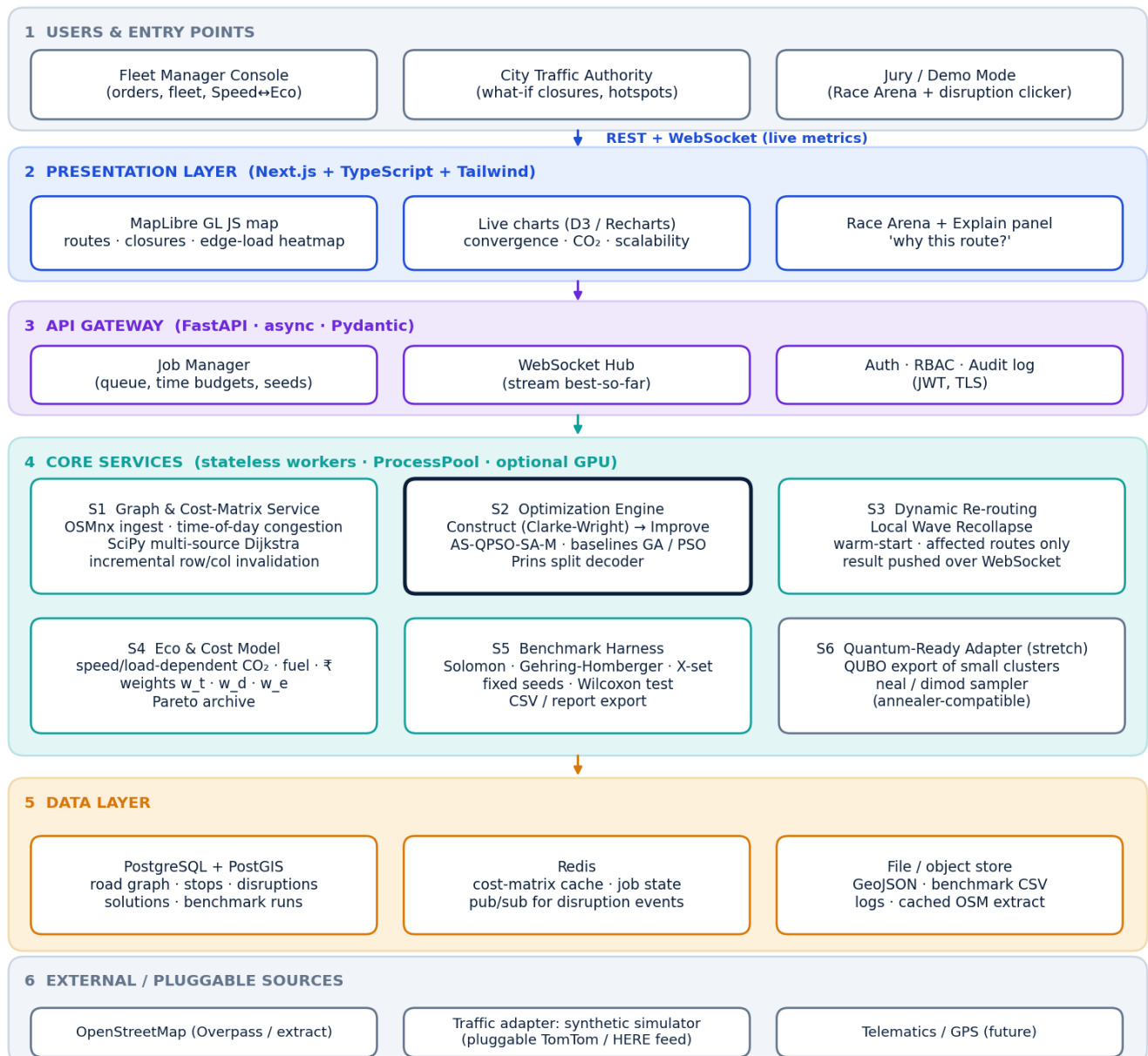


Figure 2: Layered architecture. Stateless workers run the optimisation; Redis and PostGIS keep state; the browser receives live results over WebSocket.

6.1 Layer responsibilities

Layer	Responsibility	Design notes
Presentation	Map, charts, Race Arena, explain panel, constraint toggles	Next.js + TypeScript; MapLibre GL renders thousands of route points via WebGL; charts update from WebSocket messages
API gateway	REST for commands, WebSocket for streams; job queue; auth	FastAPI async; CPU-bound optimisation runs in a worker process pool, never on the event loop; JWT + role-based access (fleet manager / authority / viewer)
Core services	Graph & cost matrix, optimisation, dynamic repair, eco model, benchmark harness	Stateless workers; every job carries a seed and a time budget for reproducibility; engine returns best-so-far at any time (anytime behaviour)
Data	Road graph, stops, disruptions, solutions, benchmark runs; cost-matrix cache; pub/sub	PostGIS for spatial data; Redis for cached matrices, job state and disruption events; files for GeoJSON and benchmark CSV
External	OSM, traffic feed, telematics	Traffic adapter interface with a synthetic simulator by default; TomTom / HERE adapters pluggable

6.2 Main API surface

Endpoint	Purpose
POST /api/v1/graphs	Load / build a road graph for an area (bounding box or place name)
POST /api/v1/jobs	Submit stops, fleet, weights, algorithm, seed and time budget; returns job id
WS /ws/jobs/{id}	Stream best-so-far cost, per-algorithm curves and interim routes
GET /api/v1/jobs/{id}/routes.geojson	Final routes with per-route distance, time, load and CO ₂
POST /api/v1/disruptions	Block or slow edges; triggers cost-matrix invalidation and Local Wave Recollapse
POST /api/v1/benchmarks	Run a benchmark suite; GET the report and raw CSV

6.3 Data model (core entities)

Edge (id, from, to, length, free-flow speed, road class, geometry, closed, delay factor, emission factor) · **Stop** (id, location, demand, time window, service time) · **Vehicle** (capacity, cost/km, emission class) · **Job** (parameters, seed, status) · **Solution** (routes, cost components, algorithm, elapsed) · **Disruption** (edges, type, start/end) · **BenchmarkRun** (instance, algorithm, seed, best, trace).

6.4 Non-functional design

- **Scalability:** horizontal scaling by adding workers; stateless API; cost matrices cached per time band; clusters of > 500 stops are decomposed cluster-first (sweep / k-means) and solved as sub-problems in parallel.
- **Responsiveness:** anytime engine with hard time budgets; interim results streamed; asynchronous I/O so map interactions never block on optimisation.
- **Reproducibility:** every run stores seed, parameters, code version and trace; benchmark reports can be regenerated bit-for-bit on CPU.
- **Security & privacy:** TLS, JWT with roles, audit log, orders stored with anonymised identifiers and coordinates only; on-premise deployment option for municipal data.
- **Deployment:** docker compose up starts web, api, worker, PostGIS and Redis; cached OSM extract and local tiles allow a fully offline demo.

7 Flow Chart

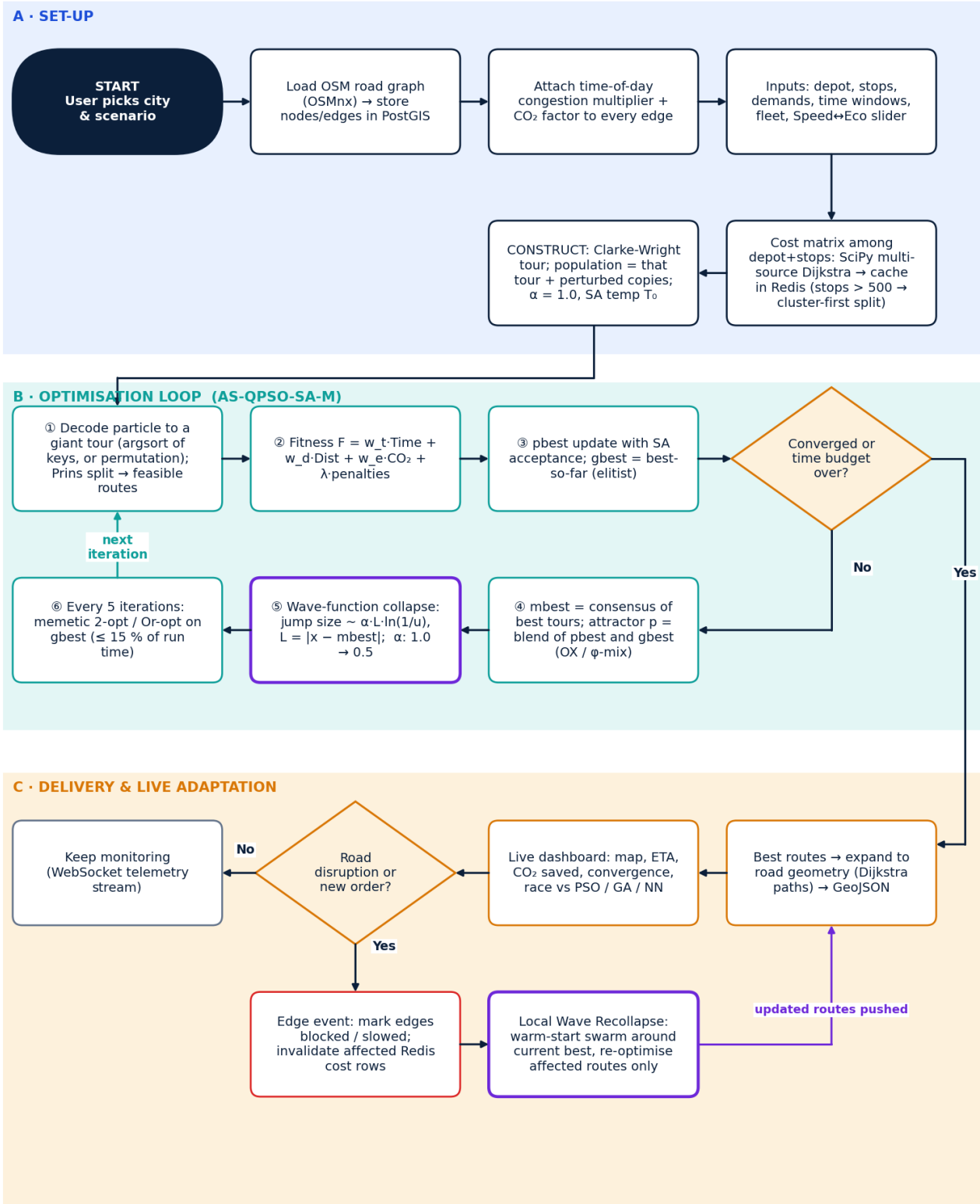


Figure 3: End-to-end flow. A: set-up and cost matrix; B: optimisation loop; C: delivery and live adaptation.

8 Technology Stack

Layer	Choice	Why this choice	Fallback
Frontend	Next.js (React) + TypeScript + Tailwind CSS	Component-based UI for sliders, toggles and live panels; fast to build a polished dashboard	Plain React + Vite
Map	MapLibre GL JS (+ deck.gl for very large layers)	Open-source WebGL map with no access token or usage quota; renders many route points smoothly	Leaflet
Charts	D3.js / Recharts	Live convergence, CO ₂ and scalability charts driven by WebSocket	Chart.js
Backend	Python FastAPI + Uvicorn + WebSockets + Pydantic	Async endpoints, typed schemas, native WebSocket streaming; same language as the engine	Flask + Socket.IO
Optimisation core	NumPy + Numba (CPU JIT); PyTorch optional for GPU batches	Numba compiled the split and local-search kernels with no GPU needed; PyTorch only pays off for very large swarms	Pure NumPy (slower)
Graph & routing	OSMnx (ingest), SciPy sparse csgraph (multi-source Dijkstra), NetworkX (analysis)	C-implemented Dijkstra for the cost matrix; NetworkX for inspection, not for bulk shortest paths	OSRM table service; igraph
Storage	PostgreSQL + PostGIS; Redis	Spatial queries and persistence; Redis for cost-matrix cache, job state and pub/sub	SQLite + files for the 36 h MVP
Baselines	Nearest-neighbour, Clarke-Wright, GA, PSO; OR-Tools as external reference	Fair, reproducible comparison behind one interface	—
Quantum-ready (stretch)	dimod + neal (simulated-annealing sampler); optional D-Wave Leap	QUBO export of small clusters with a classical sampler today	Skip in MVP
DevOps & QA	Docker Compose, GitHub Actions, pytest	One-command deployment; regression tests on benchmark instances	—

Datasets

Dataset	Use	Note
OpenStreetMap extract (an Indian metro, e.g. Bengaluru)	Road graph for the live demo	Free; requires speed / one-way sanity checks
Synthetic congestion profiles	Time-of-day edge multipliers by road class	Calibrated to public city-level congestion indices; labelled synthetic , since free edge-level history does not exist
Solomon VRPTW (C1, R1, RC1; 100 customers)	Algorithm correctness and gap to best-known solutions	Standard academic reference
Gehring & Homberger (200-1000 customers)	Scalability tests	Same format as Solomon, larger
CVRPLIB X-set (Uchoa et al.)	Capacity-only benchmarks up to 1000 customers	Best-known solutions published

9 Prototype Validation & Benchmark Plan

9.1 Method

We built the engine core as a working prototype (roughly 600 lines of Python + Numba) and tested it on synthetic capacitated routing instances: customers uniform in a 100×100 square, depot at the centre, demand 1-20, vehicle capacity 100, Euclidean distances, Prins' optimal split as decoder. All methods share the same decoder, population size (30), cost function and wall-clock budget (3, 5, 8 and 14 s for 50, 100, 200 and 400 stops) on a single CPU core; results are means over 6, 6, 4 and 3 seeds. The PSO baseline was tuned over five parameter sets on separate instances, and the hybrid was tuned on separate instances too. **These are synthetic instances, short budgets and few seeds: indicative evidence, not proof.** Section 9.5 defines the full benchmark.

9.2 Lesson 1 — unseeded metaheuristics lose to a one-second greedy heuristic

Stops	Nearest-neighbour	Clarke-Wright	GA random start	PSO random start	QPSO random start	QPSO + LS random start
50	1021	869	856	1083 (worse)	1017 (worse)	861
100	1802	1575	1647 (worse)	2734 (worse)	3303 (worse)	1775 (worse)
200	2572	2337	2711 (worse)	6765 (worse)	8680 (worse)	3087 (worse)
400	4801	4331	5736 (worse)	15967 (worse)	18665 (worse)	6295 (worse)

From 100 stops upward, **none** of the random-start metaheuristics beat the Clarke-Wright plan within the same budget (marked “worse”), and from 100 stops upward the plain random-key swarms are far worse than even nearest-neighbour dispatch. At 50 stops GA and QPSO with local search only just edge ahead of Clarke-Wright. Random-key encoding is the weak point: many key vectors decode to the same tour and nearby keys do not mean nearby tours, so the swarm's arithmetic loses meaning as n grows. **Design consequence:** never start from scratch; construct first, then improve. (Random-start runs: 3-14 s budgets; 5, 5, 4, 3 seeds.)

9.3 Lesson 2 — construct first, then improve

Every method below starts from the same Clarke-Wright tour (the population is that tour plus perturbed copies; the random-key swarms use it plus noise). Values are distance saved relative to Clarke-Wright, so **higher is better** and 0 % means no improvement over the greedy plan.

Method (all seeded with the same Clarke-Wright tour)	n = 50 (3 s, 6 seeds)	n = 100 (5 s, 6 seeds)	n = 200 (8 s, 4 seeds)	n = 400 (14 s, 3 seeds)
Nearest-neighbour + split (greedy)	-17.5 %	-14.4 %	-10.1 %	-10.9 %
Clarke-Wright savings (reference = 0 %) total distance in brackets	0.0 % (869)	0.0 % (1575)	0.0 % (2337)	0.0 % (4331)
PSO, random-key encoding	1.24 % $\pm$ 0.39	0.26 % $\pm$ 0.00	0.35 % $\pm$ 0.09	0.28 % $\pm$ 0.06
QPSO, random-key encoding (continuous mode)	0.43 % $\pm$ 0.63	0.00 % $\pm$ 0.00	0.08 % $\pm$ 0.06	0.00 % $\pm$ 0.01
GA (order crossover + inversion)	1.96 % $\pm$ 0.04	0.53 % $\pm$ 0.28	0.51 % $\pm$ 0.00	0.49 % $\pm$ 0.12
GA + local search	2.82 % $\pm$ 0.16	0.89 % $\pm$ 0.04	0.88 % $\pm$ 0.00	0.92 % $\pm$ 0.02
QI-EA: quantum-inspired jumps + SA	2.20 % $\pm$ 0.49	0.75 % $\pm$ 0.12	0.81 % $\pm$ 0.14	0.67 % $\pm$ 0.09
QI-EA-M: + local search (our engine)	2.84 % $\pm$ 0.21	1.10 % $\pm$ 0.06	0.88 % $\pm$ 0.00	1.00 % $\pm$ 0.03

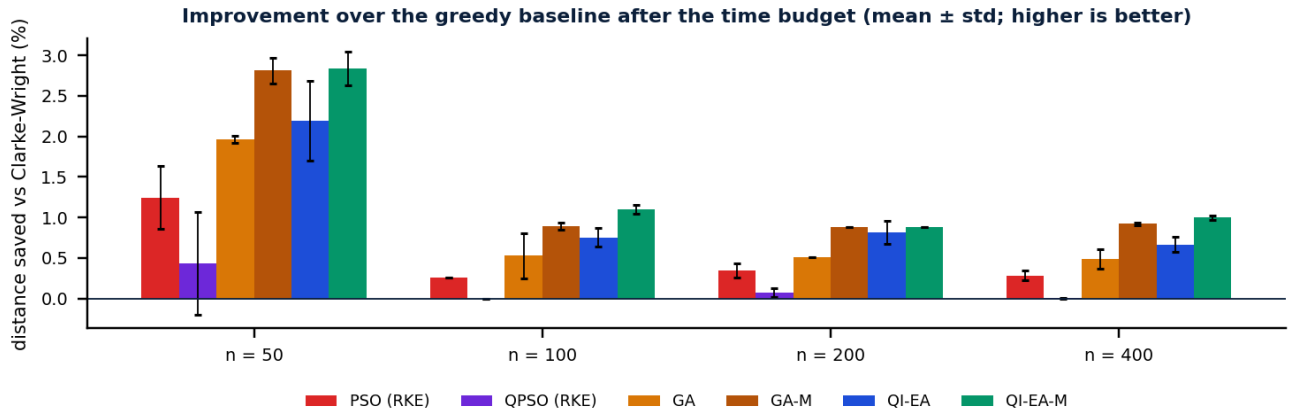


Figure 4: Improvement over the Clarke-Wright baseline after the time budget (measured; error bars show one standard deviation across seeds).

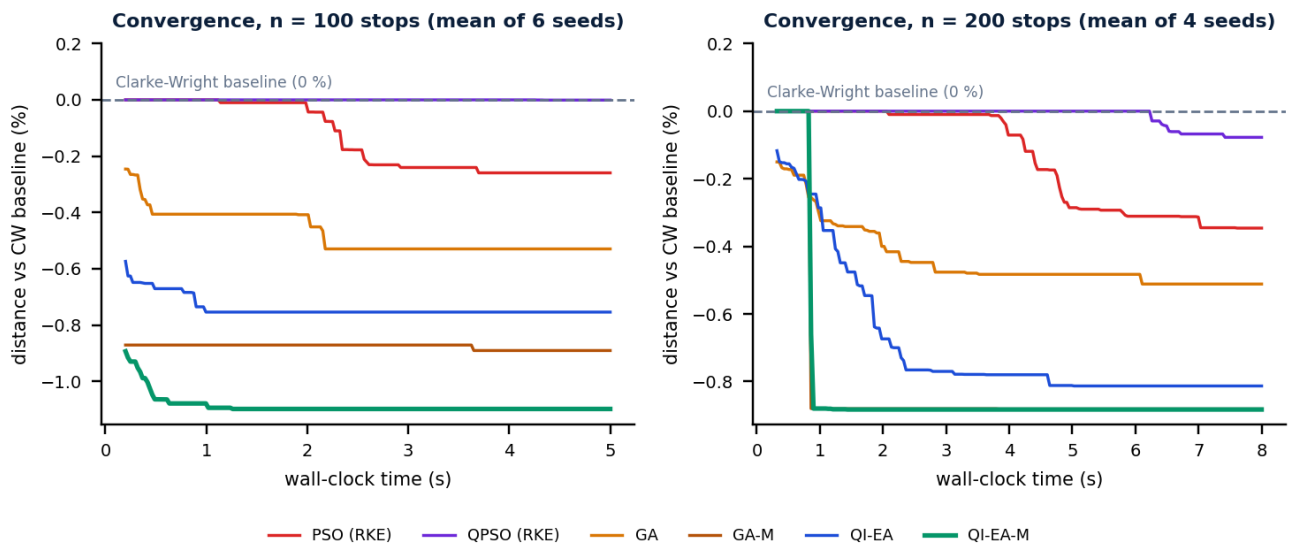


Figure 5: Convergence over wall-clock time (measured; mean over seeds). Negative values mean a shorter plan than Clarke-Wright.

9.4 What we conclude – and what we do not

What the data supports. (1) Seeding matters most: every seeded method is at least as good as its start, while unseeded ones are often much worse. (2) The improvement over a savings plan is real but modest in seconds on one core: 0.9-2.8 % for our engine (QI-EA-M) at 50-400 stops, versus 10-17 % over nearest-neighbour dispatch. (3) The quantum-inspired jump operator gave a small, consistent edge over a plain GA when neither used local search: 0.2-0.3 percentage points more distance saved, better in 4/6, 5/6, 4/4 and 2/3 seeds at 50, 100, 200 and 400 stops. (4) Local search contributes the bulk of the gain; with it the two evolutionary variants are nearly tied (equal at 50 and 200 stops, QI-EA-M ahead by about 0.2 points at 100 and 0.1 at 400). (5) Random-key QPSO, the continuous mode of the original blueprint, added almost nothing over its seed (0-0.4 %) and stayed behind random-key PSO.

What it does not support. A claim that QPSO is “superior” to GA or PSO in general, an asymptotic speed-up, or any statement about real city data. Seed counts are small (3-6), so p-values are indicative only. Our pitch therefore rests on the construct-improve-repair pipeline, the discrete quantum-inspired operator and its measured small edge, live repair and eco features, and a benchmark harness that lets anyone verify the numbers. The largest known upside is technical and already identified: delta-evaluated local search and longer budgets.

9.5 Final benchmark plan (for the hackathon and the pitch)

Element	Plan
Instances	Solomon C1 / R1 / RC1 (100 customers), Gehring-Homberger (200, 400, 600, 800, 1000), CVRPLIB X-set sample, plus a real Indian-metro OSM instance
Baselines	Nearest-neighbour, Clarke-Wright, tuned GA, tuned PSO (random-key), OR-Tools as external reference, and our ablations (with / without SA, local search, quantum-inspired jumps)
Fairness	Same machine, same cost matrix, same wall-clock budget and same seeded start for all methods; baselines tuned on separate instances
Runs & statistics	≥ 30 independent seeds per instance; report mean, std and best; Wilcoxon signed-rank test for paired comparisons; report losses as well as wins
Metrics	Total distance / cost, gap to best-known solution (%), time-to-target, convergence curve, feasibility rate, CO ₂ , re-route latency after disruption
Scalability Index	SI_q = improvement % at n_{max} divided by improvement % at n_{min} (quality retention); SI_t = exponent β of time-to-target $\propto n^\beta$ from a log-log fit (lower is better)

10 Feasibility

Dimension	Assessment	Evidence / rationale	Rating
Technical	All components are mature open-source technologies; the engine core (split, savings, GA / QPSO / hybrid, local search) already exists as roughly 600 lines of prototype code	Prototype benchmarked on 50-400 stops on a single CPU core (Section 9)	High
Algorithmic	Improvement over strong greedy is real but modest in short budgets; needs delta-evaluated local search for larger gains	Measured; mitigation plan in Section 12	Medium-High
Data	OSM and academic benchmarks are open; edge-level live traffic is not, so traffic is simulated and clearly labelled	Adapter interface keeps the door open for real feeds	High
Time (36 h)	Scope is cut into Must / Should / Could tiers with a feature freeze at hour 30	Section 10.1	Medium-High
Infrastructure	Runs on a laptop via Docker; no GPU, no paid API, offline capable	Local OSM extract, local tiles, cached matrices	High
Team & skills	Needs optimisation, backend, geospatial frontend and DevOps; a team of six covers it	Roles below	High

10.1 36-hour execution plan with cut-lines

Hours	Focus	Deliverable / exit criterion
0-6	Foundation	docker-compose up runs web + api + PostGIS + Redis; OSM graph of the demo area on the map; /health OK
6-14	Engine (Must)	Port prototype into a service: cost matrix, Clarke-Wright, GA / PSO baselines, QI-EA-M; unit tests; Solomon C101 / R101 sanity results
14-22	Integration (Must)	Job manager + WebSocket streaming; Race Arena showing live curves for all methods on one map
22-28	Live features (Must / Should)	Click-to-disrupt with Local Wave Recollapse; Speed ↔ Eco slider with CO ₂ panel; explain panel
28-33	Benchmarks & polish	Benchmark harness run on Solomon / Gehring-Homberger; scalability dashboard; UI polish. Feature freeze at hour 30
33-36	Pitch	Three full dry runs; offline fallback; pre-recorded backup of the demo

Must (MVP)	Should	Could (stretch)
Road graph, cost matrix, construct + improve engine, Race Arena, click-to-disrupt repair, benchmark table	Speed ↔ Eco slider and CO ₂ panel, explain panel, Pareto archive, scalability dashboard	Congestion-balancing layer, QUBO export, multi-stop commuter planner, GPU mode

Suggested roles (team of six): two on algorithm and benchmarks; one on backend and data; two on frontend and map; one on DevOps, integration and pitch.

11 Viability

11.1 Why it is worth adopting

- **Clear buyer value.** Fuel and driver time are among the largest controllable costs of a delivery fleet; a plan that is a few percent shorter, on time, and greener is directly measurable in rupees and tonnes.
- **Low adoption friction.** Works from a CSV of orders, needs no new hardware, runs on a small server or on-premise; dispatchers keep manual override, which builds trust.
- **Policy tailwind.** Aligned with India's National Logistics Policy (2022), PM GatiShakti (2021), the Smart Cities Mission and India's 2070 net-zero commitment; supports SDG 9, 11 and 13.
- **Defensible position.** The moat is the combination of a benchmarked engine, live repair, sustainability accounting and domain data accumulated in pilots, not a single algorithm.

11.2 Adoption path and business model

Phase	Goal	Model
1 Hackathon MVP	Prove the engine and the demo on public data	Open-source core, public benchmark report
2 Pilot (3-6 months)	Run on historical trips of one logistics partner and one municipal ITMS use case; measure real savings	Free / low-cost pilot in exchange for data access and a case study
3 Product	Multi-depot, heterogeneous fleets, live telematics, ERP / WMS integration	SaaS subscription per vehicle per month; API pay-per-optimisation; municipal what-if licence; open-core with enterprise features
4 Quantum-enabled	Offer annealer-backed solving for small hard sub-problems where it proves beneficial	Premium tier; research partnerships

11.3 Sustainability of the solution

Costs stay low because the engine is CPU-only and stateless: a pilot-scale fleet (a few hundred stops per optimisation) fits on a small virtual machine, and capacity grows by adding workers. Maintenance is limited to open-source dependencies, OSM refreshes and profile recalibration. Risks to viability — data access, integration effort and dispatcher trust — are addressed by adapters, standard file formats and the explainability panel (Section 12).

12 Potential Challenges & How We Tackle Them

#	Challenge	Why it matters	Mitigation	Fallback
1	Quantum-inspired search may not beat strong classical baselines	The jury compares against PSO / GA; our own prototype shows the margin is modest	Construct-then-improve; hybrid with local search and SA; equal-budget benchmarks with ablations; report wins and losses; delta-evaluated local search	Headline the system-level advantages (live repair, eco, transparency) and present ablations honestly
2	Continuous → discrete mapping loses quality	Random-key decoding is many-to-one and non-local; degrades with n	Discrete exponential-jump operator on permutations; optimal split decoder; seed from savings	Keep continuous mode only as a reference baseline
3	Scaling to thousands of stops	Cost matrix is n^2 ; local search is expensive	Cluster-first decomposition, k-nearest candidate lists, float32 matrices, delta evaluation, anytime time budgets, optional GPU	Report tested scale honestly; extrapolate only with labelled fits
4	Real-time repair latency	Demo and operations need answers in about a second	Warm start; affected routes only; incremental Redis invalidation; fast first repair then refine	Show progressive result: quick feasible fix, improving live
5	No free live edge-level traffic in India	Traffic-aware claims must be defensible	Synthetic time-of-day profiles by road class calibrated to city-level indices; pluggable TomTom / HERE adapter; CSV of GPS traces supported	Label all traffic as simulated; never claim real-time data
6	OSM data quality	Missing speed or one-way tags produce wrong routes	OSMnx speed imputation by road class; sanity checks; manual overrides for the demo area	Curated sub-graph for the live demo
7	Cost-matrix compute time	One Dijkstra per stop on a large graph	SciPy C-level Dijkstra; per-band caching; restrict to a bounding-box sub-graph; precompute demo matrices	OSRM table service
8	Credibility of “quantum” claims	Judges may suspect buzzwords	State plainly: quantum-inspired, runs on classical hardware; publish method, seeds and code; include OR-Tools as reference	—
9	Emission model accuracy	CO ₂ figures must be defensible	Published functional forms; per-vehicle-class coefficients; report relative savings; calibrate with telematics in the pilot	Report distance / time savings only
10	Demo and infrastructure risk	Network, tiles or laptop failure ruins a live demo	Docker offline mode; cached OSM and local tiles; health checks; pre-recorded fallback video	Run from backup machine
11	36-hour scope creep	Too many features, none polished	MoSCoW tiers; feature freeze at hour 30; engine prototype already exists; integrate continuously	Drop stretch features first
12	Privacy and trust	Fleet data is commercially sensitive; dispatchers distrust black boxes	Anonymised identifiers, RBAC, TLS, on-prem option; explain panel; manual lock-and-reoptimise	—

13 Impact & Benefits

Dimension	Benefit	How it is measured
Environmental	Less distance and less stop-and-go driving means lower fuel burn and CO ₂ ; eco-priority mode trades a little time for a lower footprint	CO ₂ per route from the emission model; relative saving vs baseline plan
Economic	Lower fuel and driver-hour cost; fewer vehicles needed for the same orders; fewer late-delivery penalties	Cost per delivery, vehicles used, on-time rate
Operational	Plans in seconds instead of hours; disruptions handled without re-planning the whole day	Optimisation time, re-route latency, plan stability (share of unchanged routes)
Urban / social	Less fleet-induced congestion; the authority can test closures before they happen	Edge-load heatmap, fleet distance in peak hours
Scientific & strategic	A reproducible benchmark harness and open method for quantum-inspired routing in India	Published benchmark report; extensibility to quantum annealers

13.1 Targets we will verify (not promises)

KPI	Prototype evidence	Target for the pilot
Distance saved vs manual / nearest-neighbour dispatch	10-17 % shorter (QI-EA-M, 50-400 stops, synthetic)	≥ 10 %
Additional saving over a savings-heuristic plan	0.9-2.8 % within 3-14 s on one CPU core	1-3 % (larger with delta local search)
Re-route latency after a disruption	Not yet measured (to be benchmarked in the integration phase)	< 1 s for up to about 100 affected stops
Time-window compliance	Handled by the split decoder (design); to be measured on Solomon	≥ 98 % of stops on time
CO₂ saving	First-order proportional to distance saved	Report relative saving per route and per fleet

13.2 Illustrative impact calculation

Assumptions (illustrative, replace with pilot data): a fleet of 100 vehicles drives 120 km per day each (12,000 km/day); light commercial diesel vehicles emit about 0.25 kg CO₂ per km. A distance reduction of 8-15 % versus manual dispatch saves **960-1,800 km per day**, i.e. roughly **240-450 kg CO₂ per day**, or about **88-164 tonnes per year** for one mid-sized fleet, before counting driver hours and fuel cost. The prototype's savings over nearest-neighbour dispatch are of this order, but this calculation is a planning example, not a measured result.

13.3 Alignment

SDG 9 (industry, innovation and infrastructure) · **SDG 11** (sustainable cities) · **SDG 13** (climate action) · National Logistics Policy 2022 · PM GatiShakti · Smart Cities Mission · Atmanirbhar software capability in quantum-inspired optimisation.

14 Roadmap & Future Scope

Horizon	Scope
Hackathon	MVP with Must and Should tiers, benchmark report, live demo
3 months	Delta-evaluated local search; heterogeneous fleets and multi-depot; ML congestion forecasting from GPS traces; pilot with one fleet
6-12 months	Driver mobile app; ERP / WMS integrations; city-level what-if platform with the traffic authority; congestion-balancing layer in production
Beyond	Annealer-backed solving of small hard sub-problems through the QUBO adapter; federated learning across fleets for privacy-preserving traffic models

R References

- [1] Sun, J., Feng, B., Xu, W. (2004). Particle swarm optimization with particles having quantum behavior. *Proc. IEEE Congress on Evolutionary Computation*.
- [2] Sun, J., Fang, W., Wu, X., Palade, V., Xu, W. (2012). Quantum-behaved particle swarm optimization: analysis of individual particle behavior and parameter selection. *Evolutionary Computation*, 20(3).
- [3] Kennedy, J., Eberhart, R. (1995). Particle swarm optimization. *Proc. IEEE International Conference on Neural Networks*.
- [4] Kirkpatrick, S., Gelatt, C. D., Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220.
- [5] Bean, J. C. (1994). Genetic algorithms and random keys for sequencing and optimization. *ORSA Journal on Computing*, 6(2).
- [6] Prins, C. (2004). A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12).
- [7] Clarke, G., Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4).
- [8] Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2).
- [9] Gehring, H., Homberger, J. (1999). A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. *Proc. EUROGEN99*.
- [10] Uchoa, E., Pecin, D., Pessoa, A., Poggi, M., Vidal, T., Subramanian, A. (2017). New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3).
- [11] Bektaş, T., Laporte, G. (2011). The pollution-routing problem. *Transportation Research Part B*, 45(8).
- [12] Lucas, A. (2014). Ising formulations of many NP problems. *Frontiers in Physics*, 2.
- [13] Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65.

A Appendix: What Changed from Blueprint v1

Blueprint v1	Issue	Correction in this document
"Exponentially wider search", "definitively prove superiority", "asymptotic time-complexity advantage"	Unsupported: QPSO has the same per-iteration complexity as PSO / GA	Empirical claims only, with equal budgets, many seeds, statistical tests and ablations (Sections 4.8, 9)
QPSO on a 10,000-node road graph	Optimiser should not run on the raw road network	Two layers: Dijkstra cost matrix among stops, optimiser on depot + stops (Section 2.2)
Attractor point undefined; SA coupling undefined	Incomplete mathematics	Full QPSO equations, α schedule and convergence bound; explicit SA and memetic steps (Section 4)
"Quantum tunnelling"	Physical analogy presented as mechanism	Exponential (Laplace) jump law with exact probabilities (Section 4.4)
Random-key encoding alone	Loses quality as n grows (measured)	Optimal split + discrete exponential-jump operator + savings seed (Sections 4.3, 4.5, 9)
"Re-routes in under 250 ms"	Unmeasured promise	Measured KPI target with warm start on affected routes (Section 4.7)
Leaflet vs Mapbox; PyTorch mandatory; PostGIS in the first 8 hours	Inconsistent and heavier than needed	MapLibre (no token); NumPy + Numba; PostGIS optional for the MVP (Section 8)
TomTom "historic indices" as congestion data	City-level, not edge-level	Synthetic profiles calibrated to indices, labelled as such (Section 8)
Solomon (100 customers) for scalability	Too small to show scaling	Add Gehring-Homberger and X-set up to 1000 customers (Sections 8, 9.5)
Dijkstra called a failing baseline	Optimal for point-to-point	Dijkstra used where optimal; optimiser only for sequencing (Section 2.2)